

ДЗ ОТ 17.09.2026 • 11 ТЭС

Система контроля версий **Git** Основы работы

Полное руководство: фундаментальная теория, история создания и пошаговый практический практикум в консоли разработчика.

ЧАСТЬ I

Теоретические основы Git

Архитектура распределенного контроля версий, предпосылки создания и фундаментальные преимущества в разработке ПО.

Что такое Git и система VCS

Что такое VCS?

VCS (Version Control System) — программное обеспечение, регистрирующее изменения в наборе файлов во времени, позволяя возвращаться к конкретным состояниям.

Она решает главную проблему хаотичной разработки: устраняет появление папок вроде «проект_финал_v2_исправлено» и ведёт хронологический аудит каждого символа кода.

Позволяет нескольким инженерам одновременно работать над общим проектом без риска стереть код коллеги.

Определение Git

Git — быстрая, распределенная (децентрализованная) система управления исходным кодом, де-факто стандарт в современной индустрии.

Ключевая особенность: в отличие от централизованных VCS (SVN, CVS), каждый разработчик хранит на локальном диске полную копию всей истории репозитория.

Операции фиксации, ветвления и просмотра истории выполняются локально без обязательного подключения к серверу.

Создатель Git: Линус Торвальдс

Имя создателя: Линус Торвальдс (Linus Torvalds) — финско-американский программист, всемирно известный создатель ядра операционной системы **Linux** (1991 г.).

Год создания Git: Апрель 2005 года. Первая рабочая версия была написана Линусом буквально за 2 недели.

Причина появления: Ядро Linux использовало проприетарную систему BitKeeper. Весной 2005 года владелец BitKeeper отозвал бесплатную лицензию у сообщества Linux.

Результат: Существовавшие альтернативы были медленными. Линус разработал принципиально новую архитектуру, ориентированную на сверхвысокую скорость и надёжность.



Ключевые преимущества Git



Высокая скорость

Все базовые операции (просмотр диффов, логов, создание коммитов) происходят локально за доли секунды без сетевых запросов.



Легковесные ветки

Ветка в Git — это лишь 41-байтовый указатель на коммит. Создание новой ветки и переключение происходят мгновенно.



Целостность данных

Все коммиты хешируются криптографическим алгоритмом (SHA-1/SHA-256). Невозможно изменить файл в истории незаметно.



Распределённость

Каждый клон — полноценный бэкап. При сбое сервера проект легко восстанавливается из репозитория любого инженера.

Где и зачем применяется Git

Зачем используется в разработке?

Хранение чистой истории: каждый коммит фиксирует кто, когда и зачем внёс конкретные строки кода.

Безопасные эксперименты: создание изолированной ветки позволяет тестировать гипотезы без риска сломать стабильный код программы.

Мгновенный откат: при обнаружении критической ошибки в продакшене код можно откатить за пару команд.

Сферы применения и интеграции

Командная разработка: параллельная работа сотен разработчиков с последующим бесконфликтным слиянием (merge).

Code Review: проверка качества чужого кода перед интеграцией на платформах GitHub, GitLab и Bitbucket.

Автоматизация (CI/CD): триггер автоматических тестов и сборки проектов при каждом пуше в ветку.

ЧАСТЬ II

Практическая работа с Git

Пошаговый консольный практикум: от создания рабочей директории до фиксации первого коммита.

Шаг 1: Создание папки проекта

Консольная навигация

Первый шаг — создание физического каталога на диске и переход в него с помощью стандартных команд ОС:

`mkdir my-project` — создает каталог.

`cd my-project` — переход внутрь папки.

Рабочая директория (Working Directory):

Это локальная папка на файловой системе, где хранятся рабочие исходные файлы, доступные для редактирования в IDE.

bash — user@workspace:~

```
user@dev:~$ mkdir my-project
```

```
user@dev:~$ cd my-project
```

```
user@dev:~/my-project$ pwd
```

```
/home/user/my-project
```

```
user@dev:~/my-project$ ls -la
```

```
total 0
```

```
drwxr-xr-x 2 user user 64 Sep 22 19:00 .
```

```
drwxr-xr-x 4 user user 128 Sep 22 18:59 ..
```

Шаг 2: Инициализация репозитория

Команда: `git init`

Превращает обычную директорию в Git-репозиторий, создавая скрытую подпапку `.git` со всей служебной базой данных.

Можно ли работать без `git init`?

Нет! Любая попытка вызвать Git в неинициализированной папке завершится критической ошибкой *`fatal: not a git repository`*, так как Git не находит конфигурацию `.git`.

bash — git init test

1. Попытка запустить Git до инициализации:

```
~/my-project$ git status
```

```
fatal: not a git repository (or any of the parent directories):  
.git
```

2. Инициализация репозитория:

```
~/my-project$ git init
```

```
Initialized empty Git repository in /home/user/my-project/.git/
```

Шаг 3: Проверка состояния

Команда: `git status`

Главный инструмент диагностики состояния репозитория. Отображает:

1. Текущую активную ветку (например, *main* или *master*).
2. Наличие коммитов в истории репозитория.
3. **Untracked files** — новые файлы, которые Git видит в папке, но ещё не отслеживает.
4. Файлы, готовые к коммиту (*staged*), либо изменённые файлы.

bash — git status

```
~/my-project$ echo 'print("Hello 11 ТЭС!")' > main.py
```

```
~/my-project$ git status
```

```
On branch main
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add ..." to include in what will be committed)
```

```
main.py
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

Шаг 4: Добавление в индекс

Команда: `git add`

Переводит файлы в **индекс (Staging Area)** — промежуточную область подготовки снимка коммита.

Способы использования:

`git add main.py` — добавление конкретного файла (точечный контроль изменений).

`git add .` — добавление ВСЕХ новых и изменённых файлов текущей директории и подпапок.

```
bash — git add main.py
```

```
~/my-project$ git add main.py
```

```
~/my-project$ git status
```

```
On branch main
```

```
No commits yet
```

```
Changes to be committed:
```

```
(use "git rm --cached ..." to unstage)
```

```
new file: main.py
```

Шаг 5: Создание коммита

Команда: `git commit -m`

Коммит (Commit) — это неизменяемый снимок (snapshot) подготовленных в индексе файлов в конкретный момент времени.

Зачем нужно сообщение (`-m "..."`)?

Сообщение коммита документирует *что именно* и *зачем* было сделано. Это позволяет всей команде понимать историю развития продукта без ручного чтения тысяч строк диффов.

bash — git commit

```
~/my-project$ git commit -m "Первый коммит"
```

```
[main (root-commit) 9a4f21d] Первый коммит
1 file changed, 1 insertion(+)
create mode 100644 main.py
```

```
~/my-project$ git status
```

```
On branch main
nothing to commit, working tree clean
```

Image Sources



<https://cdn.britannica.com/99/124299-050-4B4D509F/Linus-Torvalds-2012.jpg>

Source: www.britannica.com